

E0261 - Database Management Systems - Algorithms, Formulas, and Concepts

Suhas Kamath

November 21, 2025

Contents

1	Access Path Selection	2
1.1	Concepts	2
1.2	Formulas	3
2	Anorexic Plan Diagrams	3
2.1	Concepts	3
2.2	Algorithms	4
2.3	Formulas	4
3	Histograms	4
3.1	Concepts	4
4	Bitmap Indexes	5
4.1	Concepts	5
4.2	Formulas	5
4.3	Algorithms	5
5	Main Memory Join	6
5.1	Concepts	6
6	Columnar Indexes (Database Cracking)	6
6.1	Concepts	6
6.2	Algorithms	7
7	C-Store	7
7.1	Concepts	7
7.2	Algorithms	7
8	Data Cube	8
8.1	Concepts	8
8.2	Formulas	8
8.3	Algorithms	8
9	Data Cube View	9
9.1	Concepts	9
9.2	Algorithms	9
10	Data Mining - Apriori	9
10.1	Concepts	9
10.2	Algorithms	10
11	Data Mining - VIPER	10
11.1	Concepts	10

12 Distributed Databases	10
12.1 Concepts	10
12.2 Formulas	11
12.3 Algorithms	11
13 Join Enumeration	11
13.1 Concepts	11
13.2 Formulas	12
14 Linear Hashing	12
14.1 Concepts	12
14.2 Formulas	12
15 Memory Management	12
15.1 Concepts	12
15.2 Algorithms	13
16 Plan Bouquet	13
16.1 Concepts	13
16.2 Formulas	14
16.3 Algorithms	14
17 Database Recovery - ARIES	14
17.1 Concepts	14
18 Sampling	15
18.1 Concepts	15
18.2 Formulas	15
18.3 Algorithms	15
19 Transaction Locking	15
19.1 Concepts	15

1 Access Path Selection

1.1 Concepts

- **System R:** An experimental database management system based on the relational model of data, developed at the IBM San Jose Research Laboratory since 1975 [1].
- **SQL:** The unified query, data definition, and manipulation language used as the user interface in System R. SQL statements do not require the user to specify access paths or join order [1].
- **OPTIMIZER component:** Examines query predicates and available access paths on referenced relations, formulates cost predictions for access plans, and uses information retrieved from System R catalogs [1].
- **Access paths:** Not referenced by the user in SQL; chosen by the System R optimizer [1].
- **Clustered index:** Assumed to keep a page in the buffer long enough for every tuple to be retrieved from it [1].
- **Non-clustered index (assumption):** For relations large enough not to fit in the buffer, it is assumed that a page fetch is required for every tuple retrieval [1].
- **N-way joins (visualization):** Visualized as a sequence of 2-way joins, where a resulting composite relation is joined with the next relation [1].
- **Nested loop joins:** A method for 2-way joins that can be mixed with merge scan joins in the same query [1].

- **Merge scan joins:** A method for 2-way joins that can be mixed with nested loop joins in the same query. Often better than nested loops if the inner relation is clustered on the join column after sorting [1].
- **Intermediate composite relations:** Physically stored only if a sort is required for the next join step; otherwise, materialized one tuple at a time [1].
- **Join Order Heuristics:** Joins requiring **Cartesian products are performed as late in the join sequence as possible** [1].
- **Search tree:** Constructed to find the optimal plan for joining N relations by iteration on the number of relations joined. Stores the cheapest unordered join solution and the cheapest solution for each interesting order [1].
- **Interesting orders:** Tuple orders specified by the query block's GROUP BY or ORDER BY clauses. Every join column also defines an interesting order [1].
- **Equivalence classes for interesting orders:** Computed to minimize bookkeeping by grouping equivalent orders (e.g., if $E.DNO = D.DNO$ and $D.DNO = F.DNO$, all three columns belong to the same class) [1].
- **Tree pruning:** Computed solution costs are compared with the cheapest equivalent solution (same tables and same result order) found so far, and the cheapest solution is saved [1].
- **Selectivity factor:** Permits the optimizer to take advantage of as many restriction predicates as possible in the RSS search arguments and access paths [1].

1.2 Formulas

1. Cost Formula (General):

$$\text{COST} = \text{PAGE FETCHES} + W \times (\text{RSI CALLS})$$

W is the weighting factor between page fetches (I/O) and RSI calls (CPU utilization) [1].

2. Cost Formula for Merge Scan Inner:

$$\text{C-inner(sorted list)} = \frac{\text{TEMP PAGES}}{N} + W \times \text{RSICARD}$$

Assuming each page of the inner relation is fetched once during the merge [1].

2 Anorexic Plan Diagrams

2.1 Concepts

- **Plan Diagram:** A color-coded pictorial enumeration of the plan choices of the optimizer for a parameterized query template over the relational selectivity space [1].
- **Plan reduction (PlanRed):** The process of simplifying dense plan diagrams by reducing the number of plans without severely affecting query processing quality [2].
- **Cost increase threshold (λ):** The constraint defining plan reduction: a substitute plan's cost must be within $(1 + \lambda)$ times the optimal cost for that query point [2].
- **Anorexic Plan Diagrams:** Reduced diagrams featuring a small absolute number of plans (around **ten** is the yardstick used) [2].
- **Parametric Query Optimization (PQO):** Aims to identify the optimal set of plans for the entire relational selectivity space at compile time [1].
- **Dense plan diagram:** A plan diagram containing more than 10 plans [2].

- **Optimal Plan Reduction (PlanRed):** The problem of minimizing plan cardinality; proven to be **NP-Hard** via reduction from the Set Cover problem [2].
- **Storage-budgeted Plan Reduction Problem:** The dual of PlanRed; given a plan diagram P and a storage constraint k (retaining $\leq k$ plans), the objective is to minimize the effective cost threshold λ [2].
- **Plan Cost Monotonicity (PCM):** A condition assumed to apply to plan diagrams where cost behavior is categorized into four quadrants based on selectivity changes in X and Y dimensions [2].
- **Set Cover Problem:** A classical NP-Complete problem used to prove the NP-Hardness of PlanRed [2].
- **Query Point (q):** A point in the selectivity space, corresponding to a unique query, associated with an optimal plan P_i and a cost $c_i(q)$ [2].

2.2 Algorithms

- **Algorithm Reduce:** A polynomial-time algorithm $O(nm)$ used to convert an instance of the Set Cover problem (I') to an instance of PlanRed (G), thereby proving PlanRed is NP-Hard [2].
- **AreaGreedy:** A greedy reduction algorithm based on sorting plans by **ascending order of area coverage**. Time complexity is $O(m^2)$ [2].
- **CostGreedy:** A greedy reduction algorithm based on sorting plans by **cost**; constructs a Set Cover instance and applies the Greedy Setcover algorithm. Achieves complexity $O(nm)$ [2].
- **Greedy Setcover:** An algorithm used within CostGreedy that iteratively selects the subset covering the maximum number of remaining uncovered elements [2]. Achieves an approximation factor of $O(\ln m)$ [2].
- **ThresholdGreedy:** A greedy algorithm for the **Storage-budgeted** variant of PlanRed. Selects plans that minimize the effective cost threshold λ [2].
- **AvgEst:** A fast estimation algorithm for reduced plan cardinality. Uses the average costs of associated query points to define a simplified Set Cover instance and applies Greedy Setcover [2].

2.3 Formulas

1. **Time Complexity of Algorithm Reduce (Reduction Proof):** The polynomial time complexity of Algorithm Reduce is $O(nm)$ [2].
2. **Cost Increase Constraint:** Substitute plan cost $\leq (1 + \lambda) \times \text{Optimal Cost}$ [2].
3. **CostGreedy Approximation Factor:** $O(\ln m)$, where m is the number of query points [2].

3 Histograms

3.1 Concepts

- **Histograms:** Approximations of frequency distributions of attribute values, used by database systems for estimating query result sizes and access plan costs [3].
- **Serial histograms:** A class of histograms identified as optimal for estimating query result sizes and frequency distributions in results of single equality join queries [3].
- **End-biased histograms:** A highly practical, suboptimal subclass of serial histograms [3].
- **Physical frequency distributions:** Multi-dimensional distributions where one dimension is the relation's pages, capturing the **physical correlation** of values to pages (clustering level). Used for calculating I/O costs using an unclustered secondary index [3].
- **Logical frequency distributions:** Multi-dimensional distributions capturing **logical correlation** between attribute values [3].

- **Estimation via Multiplication:** Abstractly, estimation implies multiplying histograms in a linear algebra fashion (e.g., matrix multiplication or joins) [3].
- **Traditional Histograms:** Include equi-width and equi-depth histograms [3].

4 Bitmap Indexes

4.1 Concepts

- **Bitmap indexing:** Used for processing complex adhoc queries in read-mostly DSS environments, relying on compact bitmap sizes and efficient hardware support for Boolean operations [4].
- **Value-List index:** The simplest form of bitmap index; consists of one bitmap per attribute value [4].
- **Design Space Dimensions:** Defined by two orthogonal dimensions: **Value Decomposition** and **Bitmap Encoding Scheme** [4].
- **Value Decomposition:** How attribute values are broken into components, determining the base numbers b_i such that $C \leq \prod b_i$ [4].
- **Equality Encoding Scheme:** Uses b_i bitmaps for component i to denote equality with $0, 1, \dots, b_i - 1$ [4].
- **Range Encoding Scheme:** Uses $b_i - 1$ range bitmaps for component i , where B_i^j is set if the tuple's value $v_i < j$. Generally offers a better space-time tradeoff than equality-encoded indexes [4].
- **Space-optimal index:** Achieved with the **maximum number of components** ($n = \lceil \log_2(C) \rceil$) where all base numbers are 2 [4].
- **Time-optimal index:** Achieved with the **minimal number of components** ($n = 1$) [4].
- **Index Knee:** The index with the optimal space-time tradeoff [4].
- **Time-Optimal Bitmap Index Under Space Constraint:** The problem of finding the most time-efficient index given a maximum number of allowed bitmaps M [4].
- **Optimal Bitmap Buffering Policy:** Prioritizes buffering bitmaps from components with **smaller base numbers**, as this reduces the effective base of that component and improves time-efficiency [4].

4.2 Formulas

1. **Space-optimal Index Bitmap Count (n-component):** $n(b-2)+T$ bitmaps, where $b = \lfloor C^{1/n} \rfloor$ and T is the smallest positive integer such that $V(b-1)^{n-T} \geq C$ [4].
2. **Most Time-efficient 2-component Space-Optimal Index Base (Theorem 7.1):** $\langle b_2 - \delta, b_1 + \delta \rangle$, where $b_1 = \lceil C^{1/2} \rceil$, $b_2 = \lfloor C/b_1 \rfloor$, and $\delta = \max\{0, \lfloor \frac{b_2 - b_1 + \sqrt{(b_2 + b_1)^2 - 4C}}{2} \rfloor\}$ [4].

4.3 Algorithms

- **Algorithm TimeOptAlg:** Finds the actual optimal solution for the time-optimal index under space constraint M [4].
- **Algorithm TimeOptHeur:** A near-optimal heuristic approach to finding the time-optimal index under space constraint M [4].
- **Algorithm FindSmallestN:** Determines the smallest number of components n such that the space constraint M is satisfied by the n -component space-optimal index [4].
- **Algorithm RefineIndex:** Improves the time-efficiency of an index by maximizing the number of components with small base numbers by adjusting base number pairs. Time complexity is $O(n \log_2(n))$ [4].

5 Main Memory Join

5.1 Concepts

- **Join Optimization / Cache Consciousness:** The source illustrates performance metrics (e.g., sort-merge, simple hash, phash, radix) related to query execution and optimization, likely emphasizing cache-conscious techniques [5-16]. (Specific formal definitions are omitted in the provided excerpts).

6 Columnar Indexes (Database Cracking)

6.1 Concepts

- **Column-stores:** Physical design alternative where each attribute is stored separately, minimizing data loaded but requiring **on-the-fly tuple reconstruction** for multi-attribute queries [17, 24].
- **Late Reconstruction:** Reconstruction performed after data is selected, potentially incurring overhead if columns appear multiple times in a query [17].
- **Database cracking:** A self-organizing approach where incoming query selections continuously reorganize data storage [17].
- **Partial sideways cracking:** A self-organizing design that minimizes tuple reconstruction cost without heavy initial presorting, handling dynamic workloads and updates [17].
- **Cracker maps (M_{AB}):** Auxiliary self-organizing two-column tables (head A , tail B) materializing mappings between pairs of attributes used for tuple reconstruction [17, 24].
- **Map Set (S_A):** The collection of all cracker maps created using attribute A as the head [24].
- **Cracker index (AVL-tree):** Maintains information about how head attribute values are distributed over a cracker map M_{AB} [17].
- **Alignment:** The process of ensuring that maps within the same set S_A are physically synchronized by applying all cracking reorganizations in the **same order** [24].
- **Cracker tape (T_A):** Logs all selections on attribute A that trigger cracking of any map in S_A , ensuring adaptive alignment on demand [24].
- **Map Cursor:** Points to the entry in T_A representing the last crack applied to a specific map M_{Ax} [24].
- **Bit vector processing:** Used for multi-selection queries to filter "false candidates" that satisfy only the cracked attribute predicate, involving positional lookups [24].
- **Map Set Choice (Conjunctions):** S_A is chosen such that the restriction on A is the **most selective**, minimizing the bit vector size [24].
- **Map Set Choice (Disjunctions):** S_A is chosen based on the **least selective** attribute to minimize the area needing analysis outside the cracked region [24].
- **Partial maps:** Maps composed of independent chunks that are only partially materialized, driven by the workload and storage constraints [24].
- **Chunk map (H_A):** Contains A values and tuple keys; provides partial maps with missing chunks when necessary [24].
- **Updates (Cracking DBMS):** Updates remain pending and are applied only when a query needs the relevant data, assisting the self-organizing behavior [24, 28].

6.2 Algorithms

- **sideways.select(A, v1, v2, B):** The basic selection operator that handles creation, searching, physical reorganization ("cracking") of the relevant contiguous area w , and index updates [17].
- **sideways.select (with alignment):** Extends the basic operator by including steps to create T_A , align M_{AB} using T_A , and append the predicate $v1 < A < v2$ to T_A [24].
- **sideways.select create bv(A, v1, v2, B, v3, v4):** Creates and returns a bit vector bv for the cracked area w based on a selection predicate on attribute B [17].
- **sideways.select refine bv(A, v1, v2, B, v3, v4, bv):** Refines an existing bit vector bv using a selection predicate on attribute B [17].
- **sideways.reconstruct(A, v1, v2, B, bv):** Returns the tail values of aligned tuples in w whose bit is set in the bit vector bv [17].
- **Radix-clustering algorithm:** A cache-friendly algorithm used to improve tuple reconstruction by restricting random access to the cache, benefiting subsequent reconstruction steps [24].
- **Ripple algorithm:** Used to move pending deletes out of the result area of maps [24].

7 C-Store

7.1 Concepts

- **C-Store:** A read-optimized relational DBMS [17].
- **Column-oriented storage:** Data stored by column rather than by row [17].
- **Read-optimized Store (RS):** Component optimized for query processing [17].
- **Writeable Store (WS):** Component for handling inserts and updates [17].
- **Tuple Mover:** Component responsible for moving data from WS to RS [17].
- **Projections:** Groups of columns stored physically together and sorted on a specified sort key [17].
- **Join Indices:** Structures linking segments of different projections to reconstruct rows [17].
- **K-safety:** Physical schema design must maintain sufficient projections/join indices to handle the failure of K sites [17].
- **Storage Key (SK):** Unique ID allocated for each logical record when inserted, used to map column values back to a logical row; inferred from position in RS, physically present in WS [17].
- **Type 3 Encoding (Self-order):** A compression scheme representing every value in a sorted column as a delta from the previous value [17].
- **Snapshot Isolation:** Used for read-only transactions to avoid Two-Phase Commit (2PC) and locking [17].
- **Bitstring:** A list of zeros and ones indicating whether associated values are present in the record subset being described [17].

7.2 Algorithms

- **C-Store Query Operators:** Include Decompress, Select (σ), Mask, Project (π), Sort, Aggregate, Concat, Permute (using Join Indices), Join, and Bitstring Operators (BAnd, BOr, BNot) [17].

8 Data Cube

8.1 Concepts

- **Data cube:** The N -dimensional generalization of simple aggregate functions, used to aggregate data across multiple dimensions [29].
- **Super-aggregates:** Aggregates computed by aggregating the N -cube to lower dimensional spaces [29].
- **Rolling-up / Drilling-down:** Moving up/down the aggregation levels [29].
- **Cross-tabulation:** A 2D generalization of GROUP BY; the CUBE operator generalizes this [29].
- **ALL value:** A dummy value introduced to overload column values, denoting the set over which aggregation is computed [29].
- **GROUPING function:** A Boolean function GROUPING(A) that returns TRUE if the argument A is an ALL value [29].
- **Snowflake schema:** A general schema where aggregation granularities form a lattice [29].
- **Star schema:** Simpler schemas having a single dimension table for each dimension [29].
- **Distributive Aggregate Functions:** Computable by partitioning data, computing the function for each partition, and combining results (e.g., COUNT, SUM, MAX, MIN) [29].
- **Algebraic Aggregate Functions:** Require a bounded M -tuple scratchpad to summarize the sub-aggregation (e.g., AVERAGE, STANDARD DEVIATION). Iter_{super} function is required to fold sub-aggregates into super-aggregates [29].
- **Holistic Aggregate Functions:** No constant bound M exists on the storage needed to describe a sub-aggregate (e.g., MEDIAN, RANK, Most Frequent) [29].
- **Consistency of Aggregate Functions (CUBE Maintenance):** Aggregate functions are classified by their properties (distributive/algebraic/holistic) for SELECT, INSERT, UPDATE, or DELETE (e.g., MAX is distributive for SELECT/INSERT but holistic for DELETE) [29].

8.2 Formulas

1. **Cardinality of the resulting cube relation:**

$$\prod_{i=1}^N (C_i + 1)$$

where N is the number of attributes grouped on and C_i is the cardinality of the i^{th} attribute [29].

2. **Complexity of naive 2^N -algorithm:** Invokes the Iter function $T \times 2^N$ times, where T is the base table cardinality [29].

8.3 Algorithms

- **CUBE operator:** A relational aggregation operator that builds a table containing all 2^N super-aggregate values [29].
- **ROLLUP operator:** Produces only a linear subset of super-aggregates required for roll-up or drill-down reports [29].
- **Naive 2^N -algorithm (CUBE computation):** Iterates over input tuples, calling the Iter function 2^N times for each tuple to update all matching cells in the cube [29].
- **Optimized CUBE computation:** Computes super-aggregates from the core GROUP BY result, aggregating dimension by dimension, usually starting with the smallest cardinality dimension [29].
- **Iter_{super} function:** Iter_{super}(handle A , handle B) folds the sub-aggregate B into the super-aggregate A [29].

9 Data Cube View

9.1 Concepts

- **Materialized views:** Storing a subset of the cells (views) of the data cube to minimize query response times [30].
- **Lattice framework:** Used to model dependencies among views [30].
- **Dependence relation ($\sim$):** $Q_1 \sim Q_2$ if Query Q_1 can be answered using only the results of Query Q_2 [30].
- **Hypercube lattice:** A lattice where views are associated with a set of attributes on which grouping occurs [30].
- **Linear Cost Model:** The cost of answering a query Q is proportional to the **number of rows** (size) of the materialized ancestor view Q_A chosen to answer Q [30].
- **Optimal Materialization (Complexity):** The problem of selecting materialized views to minimize average query cost under a space constraint is **NP-complete** (via reduction from Set-Cover) [30].
- **Benefit ($B(v, S)$):** The heuristic measure used in the greedy algorithm. The benefit of view v relative to the materialized set S is the sum of cost improvements it offers across all views $w \leq v$ [30].
- **Greedy Algorithm Performance Guarantee:** The algorithm performs within a small constant factor of optimal. If weighted by query frequency, the benefit achieved is at least $\approx 63\%$ of the optimal benefit [30].
- **Materialization Order:** A simple descending-order topological sort on the $\sim$ operator determines the required order of materialization [30].

9.2 Algorithms

- **Greedy Algorithm for Materialized Views:** Iteratively selects the view v (not already in the materialized set S) that maximizes the benefit $B(v, S)$ [30].

10 Data Mining - Apriori

10.1 Concepts

- **Association Rule:** An implication $X \implies Y$, where X and Y are disjoint sets of items [31].
- **Support (s):** The percentage of transactions containing $X \cup Y$. Must exceed minimum support (minsup) [31].
- **Confidence (c):** The percentage of transactions containing X that also contain Y . Must exceed minimum confidence (minconf) [31].
- **Large itemsets (L_k):** Sets of k items that have transaction support above minimum support [31].
- **Candidate itemsets (C_k):** Potentially large itemsets generated from L_{k-1} [31].
- **AIS / SETM algorithms:** Older algorithms that generate and count candidate itemsets **on-the-fly** during the database scan [31].
- **Apriori approach (Candidate generation):** Generates candidate itemsets C_k **before** the database scan, using the large itemsets L_{k-1} [31].
- **AprioriTid set (C_k):** The set of candidate k -itemsets when the TIDs of the generating transactions are kept associated with the candidates. Used by AprioriTid to count support after the first pass, avoiding subsequent database scans [31].
- **Itemset Representation:** $c \cdot c \cdots c[k]$ for a k -itemset c , where items are kept in lexicographic order [31].

10.2 Algorithms

- **Apriori algorithm:** Iterative algorithm where, in pass k , it uses L_{k-1} to generate C_k via apriori-gen, then scans the database to count support [31].
- **AprioriTid algorithm:** Uses apriori-gen but relies on the set C_{k-1} from the previous pass to count support for C_k (eliminates database scans after pass 1) [31].
- **AprioriHybrid:** Combines Apriori and AprioriTid, using Apriori initially and switching to AprioriTid when the intermediate set C_k is expected to fit in memory [31].
- **apriori-gen function:** Generates candidate itemsets C_k from L_{k-1} by performing a **join step** (joining L_{k-1} with itself) and a **prune step** (deleting candidates whose $(k-1)$ -subsets are not in L_{k-1}) [31].
- **subset function:** Used by Apriori to efficiently determine candidates in C_k contained within a given transaction t . Candidates are stored in a **hash-tree** [31].

11 Data Mining - VIPER

11.1 Concepts

- **i-length snakes:** A structure used in a data mining computation flow, involving dynamic generation and reading/writing to disk [18].

12 Distributed Databases

12.1 Concepts

- **Distributed Database System:** Data resides in several locations; components are loosely coupled and share no physical components [32].
- **Multidatabase Systems (MDBS):** Software built on top of heterogeneous collections of existing database systems [32].
- **Replication:** Storing identical copies (replicas) of a relation at different sites to increase availability and retrieval parallelism [32].
- **Fragmentation:** Partitioning a relation into several fragments, stored at different sites [32].
- **Horizontal Fragmentation:** Partitioning based on a selection predicate ($\sigma_p(r)$) [32].
- **Vertical Fragmentation:** Partitioning based on a projection of attributes ($\pi_{R_i}(r)$), typically requiring a **tuple-id** attribute for reconstruction [32].
- **Mixed Fragmentation:** Applying both horizontal and vertical fragmentation schemes [32].
- **Network Transparency:** Hiding details of data location, naming, replication, and fragmentation from users [32].
- **Distributed Transaction Model:** Global transactions are executed by a coordinator across multiple sites, requiring ACID properties [32].
- **Two-Phase Commit Protocol (2PC):** Ensures atomicity of global transactions. Phase 1: prepare T ; Phase 2: commit T or abort T [32].
- **Three-Phase Commit Protocol (3PC):** Designed to avoid blocking failures in a restricted class of failures by adding a precommit phase [32]. Requires $K \geq 1$ sites to be up [32].
- **Locking Protocols (Distributed):** Includes Single-Manager Approach, Multiple Coordinators, Majority Protocol, and Biased Protocol [32].
- **Majority Protocol:** Requires T to obtain a lock on a majority of replicas of data item Q [32].

- **Biased Protocol:** Favors shared locks (requiring only one replica lock) over exclusive locks (requiring all replica locks) [32].
- **Deadlock Handling (Distributed):** Requires detecting cycles in a **global wait-for graph** [32].
- **Two-Level Serializability (2LSR):** Ensures serializability at both the local DBMS level and the global MDBS level [32].

12.2 Formulas

1. **Horizontal Fragmentation Reconstruction:** $r = r_1 \cup r_2 \cup \dots \cup r_n$ [32].
2. **Vertical Fragmentation Reconstruction:** $r = r_1 \bowtie r_2 \bowtie \dots \bowtie r_n$ [32].

12.3 Algorithms

- **Semijoin Strategy:** Optimization technique used in distributed query processing to reduce data transfer by transmitting only necessary attributes and tuple IDs [32].
- **Naming-translation algorithm:** Function to map names/fragments to their physical locations and/or replicas [32].
- **Coordinator-Failure Protocol (3PC):** Protocol triggered when a site fails to hear from the coordinator, leading to the selection of a new coordinator using an election algorithm [32].

13 Join Enumeration

13.1 Concepts

- **Join Enumeration:** The process of choosing the "best" sequence of 2-way joins to achieve an N -way join [33].
- **Dynamic Programming:** Technique used by System R/R* to evaluate join sequences [33].
- **Left-deep join trees:** Plans constructed by joining a single table with the composite result of previous joins (e.g., $((T_1 \bowtie T_2) \bowtie T_3) \bowtie T_4$) [33].
- **Bushy trees / Composite Inner:** Plans where the inner operand of a join is the result of a previous join; these results must be materialized [33].
- **Cartesian Products (in Join Enumeration):** Joins between tables lacking a join predicate. Often deferred until late in the sequence [33].
- **Starburst Plan Optimization:** Consists of the **join enumerator** and a **rule-based plan generator** [33].
- **Query Graph Model (QGM):** Internal representation of the semantics of an SQL query, storing entities like tables and quantifiers [33].
- **Quantifier:** Corresponds to a join variable, representing a tuple drawn from a table [33].
- **Starburst Join Enumerator:** Uses a **generate and filter** strategy with mandatory and **optional parameterized heuristics** to control the search space [33].
- **Linear Query Shape:** Tables connected by binary predicates in a straight line. Optimization complexity is **polynomial** [33].
- **Star Query Shape:** Quantifiers connected to a central hub quantifier. Optimization complexity is **exponential** [33].
- **Parameterized Feasible Join Criterion:** Used to control the maximum size of smaller quantifier sets in a join [33].

13.2 Formulas

1. **Complexity of Linear Queries with Composite Inners (Feasible Joins):** $\frac{N^3-N}{6}$ feasible joins [33].
2. **Complexity of Linear Queries without Composite Inners (Feasible Joins):** $\frac{N^2-N}{2}$ feasible joins [33].
3. **Complexity of Star Queries (Feasible Joins):** $(N-1)2^{N-2}$ feasible joins [33].

14 Linear Hashing

14.1 Concepts

- **Hashing (General):** Technique addressing records by primary key c , mapping it via a hash function h to a bucket $h(c)$. Allows near one-access retrieval for static files [34].
- **Dynamic Hashing:** Techniques required when files are dynamic to maintain good performance, avoiding rehashing the entire file [34].
- **Linear Hashing (LH):** A dynamic hashing scheme that requires only a few bytes of core memory regardless of file size [34].
- **Split order (LH principle):** The main new idea: split addresses are chosen in a **predefined (linear) order** instead of splitting the overflowing bucket [34].
- **Level (j) and Pointer (n):** Variables used in LH to track the split process [34].
- **Load control:** A variant of the basic LH schema where splits are controlled by the file load [34].

14.2 Formulas

1. **Hashing Function Example:** $h_j(c) = c \bmod M$; $M = 2, 3, \dots$ [34].
2. **Physical Address Formula (Totally distributed allocation, $q = 1$):** $m'(m) = T(m)$, where T is a table of non-contiguous quantum addresses [34].

15 Memory Management

15.1 Concepts

- **Page Replacement Policy:** The decision of which current buffer page should be dropped to make space for a new page [35].
- **LRU (Least Recently Used):** Conventional policy that drops the page accessed longest ago [35].
- **LRU Limitation:** Uses only the time of the last reference, failing to differentiate between frequently and infrequently referenced pages [35].
- **LRU-K Method:** New buffering method that tracks the times of the **last K references** to popular pages to statistically estimate interarrival times [35].
- **Interarrival times (I_p):** The time between successive references to a page p . LRU-K uses the backward K-distance to estimate this statistically [35].
- **AO Algorithm (Optimal Algorithm):** The theoretical optimal buffering algorithm that replaces the page with the lowest probability of future reference (maximum expected interarrival time) [35].
- **Backward K-distance ($b_t(p, K)$):** The time of the K^{th} most recent non-correlated reference to page p [35].

- **Independent Reference Model (IRM):** Statistical model for paging used to justify the LRU-K approach [35].
- **Correlated Reference Period (Time-Out Correlation method):** A short time period during which successive references are assumed correlated. LRU-K calculates interarrival time based on non-correlated access pairs [35].
- **HIST(p) (History Control Block):** Data structure storing the times of the K most recent non-correlated references to page p [35].
- **LAST(p):** Time of the most recent reference to page p , correlated or not [35].

15.2 Algorithms

- **LRU-K Buffering Algorithm (Schematic Outline):** Details the update procedure for HIST(p) and LAST(p) upon a page reference, specifically incorporating the Correlated_Reference_Period check [35].

16 Plan Bouquet

16.1 Concepts

- **Plan Bouquet Approach (BOU):** A robust execution method that eschews compile-time selectivity estimation for error-prone selectivities, instead **discovering** them at run-time through cost-limited plan executions [36].
- **Parametric Optimal Set of Plans (POSP):** The set of optimal plans covering the selectivity range of an error-prone predicate [36].
- **POSP Infimum Curve (PIC):** The trajectory of the minimum cost achieved among all POSP plans; represents ideal performance [36].
- **Isocost (IC) steps:** Graded progression of cost thresholds (often geometric, e.g., doubling) projected onto the PIC to discretize the solution space [36].
- **Plan Bouquet (B):** The subset of POSP plans associated with the intersections of the isocost steps with the PIC [36].
- **Robustness / Maximum Sub-Optimality (MSO):** Robustness is the worst-case sub-optimality in plan performance. MSO is the worst-case ratio of observed cost to optimal cost [36].
- **Cost-doubling strategy:** The choice of common ratio $r = 2$ for geometric progression of IC steps, which yields the minimal MSO bound in the 1D scenario [36].
- **AxisPlans:** The set of bouquet plans present at the **intersection** of the current isocost contour with the dimensional axes corresponding to the running location q_{run} as origin [36].
- **Contour-focused approach:** A compile-time method for generating POSP by recursively dividing hypercubes only through which isocost contours pass [36].
- **First Quadrant Invariant:** A run-time strategy ensuring that the actual selectivity location (q_a) is always in the first quadrant relative to the running selectivity location (q_{run}), supporting monotonic progress [36].
- **Manhattan Profile:** The movement of q_{run} following a profile resembling Manhattan distance due to focusing on learning only a single selectivity in each plan execution [36].

16.2 Formulas

1. **MSO (1D Scenario, Geometric Progression r):**

$$\text{MSO} \leq \frac{r^2}{r-1}$$

The minimum value is $\text{MSO} \leq 4$ when $r = 2$. This is the best competitive factor achievable by any deterministic algorithm [36].

2. **MSO (Multi-D Scenario):** $\text{MSO} \leq \rho \frac{r^2}{r-1}$, where ρ is the maximum contour plan density [36].

16.3 Algorithms

- **Basic Bouquet Algorithm (Run-time):** Discovers query location by sequentially executing the set of plans on each IC contour in a cost-limited manner until one completes [36].
- **Optimized Bouquet Algorithm (Run-time):** Incorporates explicit selectivity tracking, the AxisPlan routine for prioritizing plans, and update logic for q_{run} to maximize selectivity movement [36].

17 Database Recovery - ARIES

17.1 Concepts

- **ARIES (Algorithm for Recovery and Isolation Exploiting Semantics):** A recovery method supporting fine-granularity locking, partial rollbacks, and Write-Ahead Logging (WAL), designed for steal/no-force buffer policies [37].
- **Write-Ahead Logging (WAL):** Protocol ensuring log records describing data changes reach stable storage before the changed data is written to nonvolatile storage [37].
- **Log Sequence Number (LSN):** Unique, monotonically increasing identifier assigned to every log record. Used in every page (page.LSN) to correlate the page state with logged updates [37].
- **Log (Trail or Journal):** History of actions executed by the DBMS, stored on stable storage [37].
- **Compensation Log Record (CLR):** Written during rollbacks to log changes made during undo. CLRs contain UndoNxtLSN and are never undone themselves [37].
- **Repeating History:** The ARIES principle during the Redo phase: all missing updates (committed and loser transactions) are reapplied to restore the database to its state just before the failure [37].
- **Fuzzy Checkpoint:** Checkpoint taken asynchronously without quiescing transactions or forcing dirty pages to disk, making it inexpensive [37].
- **ARIES Recovery Phases:** 1. **Analysis** (determines RedoLSN, dirty pages, active transactions). 2. **Redo** (repeats history). 3. **Undo** (rolls back loser transactions using CLRs) [37].
- **RedoLSN:** The log location where the Redo pass starts [37].
- **RecLSN (Recovery LSN):** Stored in the Dirty Page Table; indicates the log point from which updates might not yet be in nonvolatile storage [37].
- **UndoNxtLSN:** Field in CLRs, pointing to the next non-CLR record to be undone, ensuring bounded logging [37].
- **Logical Undo:** The ability during undo to affect a page different from the one modified during forward processing, supporting higher concurrency in index structures [37].
- **Nested Top Actions:** Actions treated as atomic database operations that commit regardless of the transaction's fate; logged using redo-only records or dummy CLRs [37].

- **Latches:** Short-lived synchronization primitives guaranteeing **physical consistency** in shared memory [37].
- **Shadow Page Technique (System R):** Recovery method based on action-consistent versions of the database; superseded by WAL-based techniques due to various drawbacks [37].

18 Sampling

18.1 Concepts

- **Sampling-based algorithms:** Methods for query size estimation that do not rely on assumptions about probability distributions or detailed pre-stored statistics [38].
- **Adaptive Random Sampling Algorithm:** Algorithm for size estimation based on conceptually **partitioning** the query answer into disjoint subsets [38].
- **Partitioning (Query estimation):** Conceptual division of the query answer space into disjoint subsets [38].
- **Termination Condition (Adaptive Sampling):** Expressed in terms of the size of the sum of the samples taken (rather than the number of samples), making it adaptive [38].
- **Sanity bounds:** Proposed to handle highly skewed data, guaranteeing that the estimate is accurate or that the true query size is small [38].
- **Source Relation / Target Relation (Join Sampling):** For binary joins, the source relation is partitioned, and the size of a sample is the number of tuples in the target relation that join with a sampled tuple from the source [38].
- **Block Sampling (Selectivity):** A variant where a sample is a randomly chosen disk page, and the sample size is the number of tuples on that page satisfying the selection [38].

18.2 Formulas

1. **Estimation Result (A):**

$$A = n \times s/m$$

where n is the total number of partitions, s is the sum of sample sizes, and m is the number of samples taken [38].

2. **Error bound condition:** Error in A is less than A/d with probability p , provided a quantity derived from A meets a bound k_1/error where $k_1 = 1/(1 - \sqrt{p})$ [38].

18.3 Algorithms

- **General Algorithm for Query Size Estimation:** A loop that repeatedly takes samples S_i by randomly choosing a subset, computes the sample size S_i , and iterates until the sum of samples s meets the termination condition [38].

19 Transaction Locking

19.1 Concepts

- **Lock Granularity Hierarchy / DAG:** Resources organized hierarchically or as a Directed Acyclic Graph (DAG) for locking [39].
- **Intention Mode (I):** Used to "tag" all ancestors of a node to signal that finer-level locking (S or X) is occurring below [39].
- **Access Modes:** NL (Null), IS (Intention Share), IX (Intention Exclusive), S (Share), SIX (Share Intention Exclusive), X (Exclusive) [39].

- **Lock Protocol Order (Hierarchy):** Locks requested **root to leaf** and released **leaf to root** [39].
- **Lock Protocol Order (DAG):** To lock a node, one must lock **all the parents** of the node in the DAG in the appropriate intention mode [39].
- **Index Interval Locks / Phantoms:** Making each key value in an index a lockable object is a form of predicate locking used to solve the problem of **phantoms** [39].
- **Well Formed Transaction:** Locks an entity in exclusive (shared or exclusive) mode before writing (reading) it [39].
- **Two Phase Locking (2PL):** Transaction acquires locks (growing phase) before releasing them (shrinking phase) [39].
- **Degree 3 Consistency:** Highest level: T is **well formed** and **two phase** [39].
- **Degree 2 Consistency:** T is **well formed** and **two phase with respect to writes** [39].
- **Degree 1 Consistency:** T is **well formed with respect to writes** and **two phase with respect to writes** [39].
- **Degree 0 Consistency:** Lowest level: T is **well formed with respect to writes** [39].
- **Dependency Relations ($<$, $\ll$, $\lll$):** Relations defined by the sequence of Write (W) and Read (R) actions between transactions [39].
- **Consistent Schedule:** A schedule is degree i consistent if the transitive closure of the corresponding dependency relation ($<^*$, $\ll^*$, $\lll^*$) is a partial order [39].